

ECE 310

Project 1 Report: 8x8 Dadda Unsigned Multiplier

Alex Chapman

September 5 2026

Abstract

When an engineer designs hardware, the solutions for addition and subtraction come rather naturally. Full and ripple-carry adders are well-known designs that efficiently handle the computation. When one considers multiplication, it becomes a greater challenge. A Dadda Tree Multiplier is a hardware component invented in 1965 by computer scientist Luigi Dadda. It multiplies two unsigned binary numbers into a list of partial products, then uses Full and Half Adders to reduce the partial product "matrix" to a two-row matrix to be inserted into a ripple-carry adder. For Project 1, we are implementing an 8-bit by 8-bit Dadda multiplier that takes two 8-bit unsigned inputs, `a` and `b`, and produces a 16-bit unsigned product, `prod`. The implementation involves generating a full matrix of 64 partial products using AND gates, compressing that matrix down to two rows using Half Adders and Full Adders across successive reduction stages, and finally combining the two remaining rows using a 14-bit ripple-carry adder to produce the final product.

1 Introduction

Digital circuits are underpinned by logic design, a mathematical framework that enables vast computation to be carried out through logic gates. Typically, multiplication can be quite hardware-intensive, in both calculation and internal net size. We must remember that unlike software, an extra Verilog variable costs actual silicon. The Dadda multiplier addresses this by reducing the height of the partial product matrix to a specified height using a scheduled sequence of Half and Full Adders. This differs from a Wallace multiplier, which reduces the height of the matrix as much as possible in each stage; a Dadda multiplier only reduces the height enough to meet a specific target height at each stage, resulting in a comparable number of reduction stages, but with fewer total adders and a more regular structure [1].

For this project, students designed and implemented an 8x8 unsigned Dadda multiplier in gate-level Verilog, including the gate-level Half Adder (HA) and Full Adder (FA) modules. A testbench was then developed to apply a range of non-trivial input combinations and confirm that the circuit's output matches the expected results of standard binary multiplication.

2 Design & Implementation

2.1 Partial Product Generation

The first stage of the multiplier generates the full 8x8 partial product matrix using AND gates, where each partial product `partialProductMatrix[i][j]` is the logical AND of bit `a[i]` and bit `b[j]`. Row `j` of the matrix corresponds to a fixed bit of `b`, while column `i` corresponds to a fixed bit of `a`. This produces the full parallelogram arrangement of 64 partial products, matching the design diagram in Figure 1.

Listing 1: Sample partial product generation (Part of Row 0 shown; Rows 1-7 follow the same pattern).

```
1 and a_A2_B0(partialProductMatrix[0][2], a[2], b[0]);
2 and a_A3_B0(partialProductMatrix[0][3], a[3], b[0]);
3 and a_A4_B0(partialProductMatrix[0][4], a[4], b[0]);
```

If a partial product does not need to be reduced at any stage, it will be stored in the vectors `ppFinalA/ppFinalB`, saving silicon, as noted before.

2.2 Reduction Stages

Following the classic Dadda reduction sequence[1], the initial matrix height of 8 was reduced in stages targeting heights of 6, 4, 3, and finally 2, using the sequence $d_{j+1} = \lfloor 1.5 d_j \rfloor$ worked backward from the final target height of 2. At each stage, any column exceeding the target height for that stage was reduced using Half Adders (for columns with exactly two excess bits) and Full Adders (for columns with three or more bits needing compression), consistent with the project requirement to prefer a Half Adder whenever only two bits require summing.

Listing 2: Gate-level Full Adder (FA) and Half Adder (HA) submodules.

```
1 module FA(
2     input A,B,Cin,
3     output Cout, S
4 );
5     wire xor1_out, and1_out, and2_out;
6     xor x1(xor1_out, A, B);
7     xor x2(S, xor1_out, Cin);
8     and a1(and1_out, A, B);
9     and a2(and2_out, xor1_out, Cin);
10    or o1(Cout, and1_out, and2_out);
11 endmodule
12
13 module HA(
14     input A,B,
15     output Cout, S
16 );
17     xor x1(S, A, B);
18     and a1(Cout, A,B);
19 endmodule
```

2.3 Final Addition

After all the reduction stages are complete, the original 8x8 partial product matrix is now two rows of bits (stored as `ppFinalA` and `ppFinalB`), to be put into a 14-bit ripple-carry adder. This RCA is built from one Half Adder (for the LSB position of that stage, since no carry-in exists yet) followed by 13 cascaded Full Adders, to produce the final product bits `prod[14:1]` and the overall carry-out `prod[15]`. This produces the intended 16-bit `prod` value in the spec.

Listing 3: Final combination of the two reduced rows using a 14-bit ripple-carry adder.

```
1 buf(prod[0], ppFinalA[0]); //Due to the tree shape, prod[0] is the result
  of just ppFinalA[0]
2 rca_14bit ripple1(ppFinalA[14:1], ppFinalB[14:1], prod[14:1], prod[15]);
```

2.4 Gate Count Summary

Table 1 summarizes the total number of gate-level units used across the entire design, including both the Dadda reduction tree and the final ripple-carry addition stage.

Unit	Count
Total Dadda Half Adders	7
Total Dadd Full Adders	35
Total Half Adders	8
Total Full Adders	48

Table 1: Gate-level counts for the 8x8 Dadda multiplier.

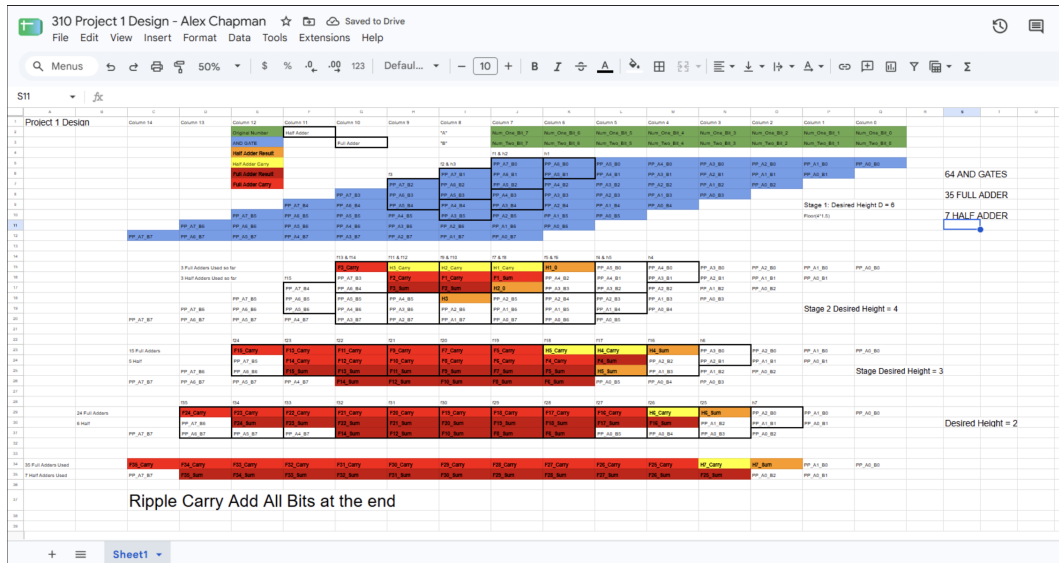


Figure 1: Project 1 Design Phase: showing the progression from height=8 to height=2 before insertion into a ripple-carry adder.

3 Simulation

The Dadda multiplier was verified using a Verilog testbench that drove the **a** and **b** inputs with several non-trivial combinations of values and automatically compared the resulting **prod** output against the expected product computed with Verilog’s built-in multiplication operator. Inputs were chosen to exercise the full width of the partial product matrix, edge cases at the extremes of the input range, and general non-trivial values.

3.1 Test Vectors Used

- **a** = 1111_1111, **b** = 1111_1111 → **prod** = 1111_1110_0000_0001 ($255 \times 255 = 65025$) tests the "all ones" edge case, making sure every column of the partial product matrix at maximum height ends up correct, and confirming correct carry propagation through the entire reduction tree.
- **a** = 1010_1010, **b** = 0101_0101 → **prod** = 0011_1000_0111_0010 ($170 \times 85 = 14450$) tests an alternating bit pattern, which produces a partial product matrix with a diagonal structure and utilizes a unique set of adders than the previous.
- **a** = 0000_1111, **b** = 1111_0000 → **prod** = 0000_1110_0001_0000 ($15 \times 240 = 3600$) tests a 4-bit alternation, where only the upper half of **b** and the lower half of **a** are set, making sure that partial products are correctly zeroed out and do not falsely pass on carries.
- **a** = 1001_1100, **b** = 0110_1101 → **prod** = 0100_0010_0110_1100 ($156 \times 109 = 17004$) tests a normal operation case, with multiple internal carries.
- **a** = 0000_0000, **b** = 1111_1111 → **prod** = 0000_0000_0000_0000 ($0 \times 255 = 0$) tests the "all zeroes" edge case to make sure no partial product incorrectly propagates when it should not.
- **a** = 0000_0001, **b** = 0000_0001 → **prod** = 0000_0000_0000_0001 ($1 \times 1 = 1$) tests the smallest non-zero case, confirming the unique path that **ppFinalA[0]** takes (buffered through to the end) and the ripple-carry adder’s least-significant stage.

3.2 Results

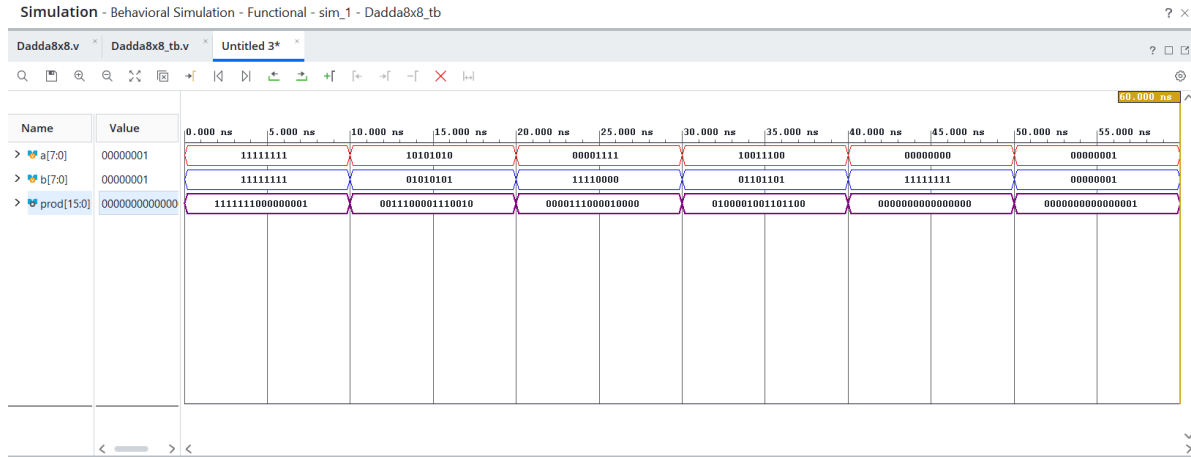


Figure 2: Output Waveforms generated by all 6 test vectors

All six test cases produced a `prod` output that matched the expected result of standard unsigned binary multiplication, with no mismatches reported by the testbench. This confirms that the partial product generation, the staged Half Adder/Full Adder reduction network, and the final ripple-carry addition all function correctly together, and that the reduction schedule successfully compresses the matrix from an initial height of 8 down to 2 rows before the final addition, consistent with the Dadda reduction approach.

4 Conclusions

The Dadda multiplication algorithm is an extremely efficient multiplication strategy in hardware, especially when we are concerned with the physical size of our silicon. After testing, results confirmed the gate-level 8x8 implementation of the Dadda Multiplier produced expected outputs based on a non-trivial number of test vectors. This project demonstrates to ECE310 students that a properly planned hardware component can save an engineer precious silicon. As this project currently stands, one could consider an implementation that includes an adder with a smaller propagation delay, to take this project further. An engineer that considers all parts of an algorithm/project would want to eliminate the propagation delay that a ripple-carry adder intrinsically has, and might consider a look-ahead adder or other implementation.

References

- [1] "Dadda multiplier," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Dadda_multiplier
- [2] Stianmat, "Dadda tree 8x8," Wikimedia Commons, 5 January 2009, licensed under CC BY 3.0. [Online]. Available: https://commons.wikimedia.org/wiki/File:Dadda_tree_8x8.svg

This report was compiled using L^AT_EX.